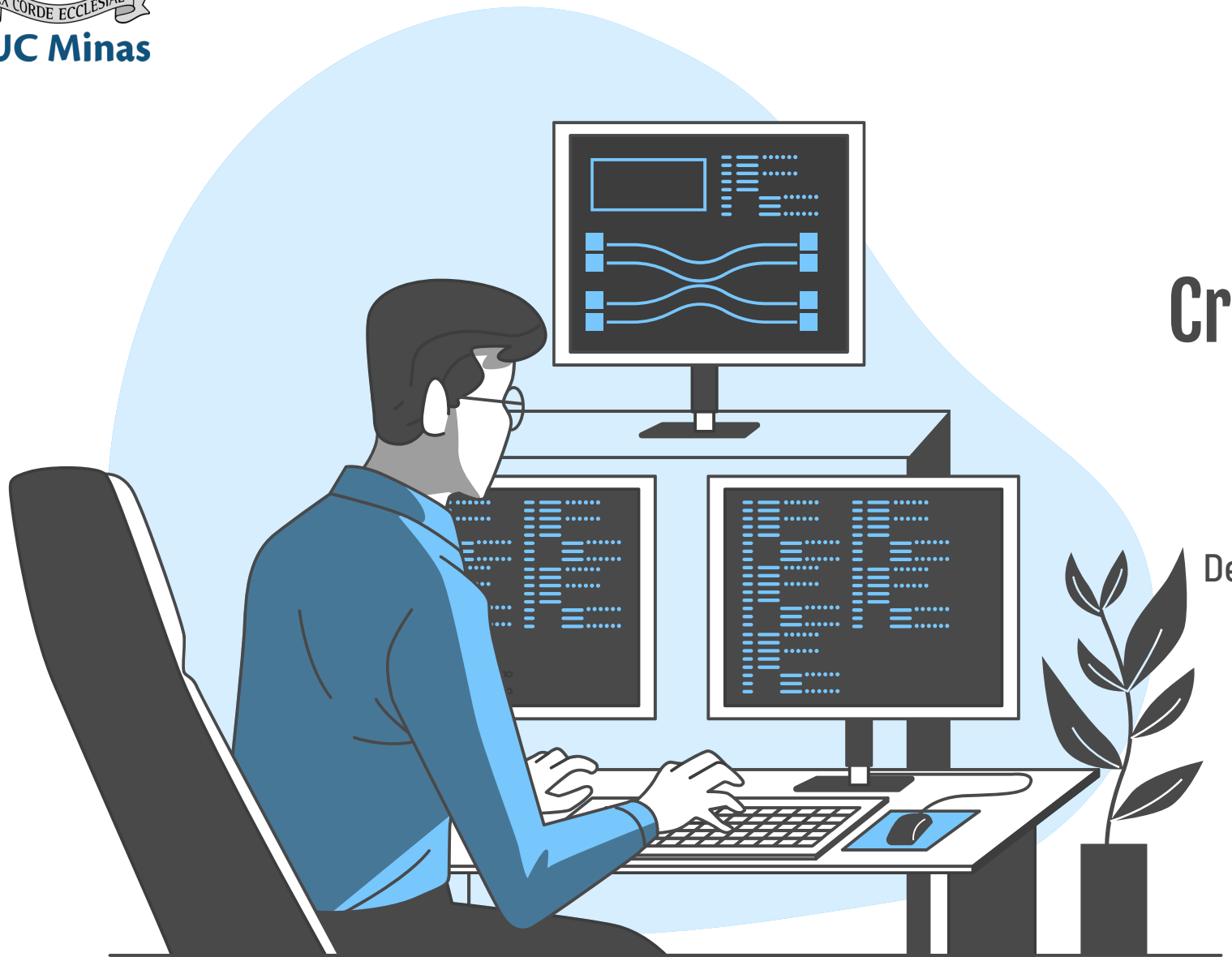
PUC Minas



Aula 02 · Back-end

Criando um projeto Spring Boot

Desenvolvimento e Integração de Aplicações Web

Prof. Dr. João Paulo Aramuni

Engenharia de Software · PUC Minas

github.com/joaopauloaramuni/desenvolvimento-e-integracao-de-aplicacoes-web

Apresentação da aula

Nesta aula você vai

1. Gerar um projeto pelo **Spring Initializr** (site e VS Code)
2. Escolher as **dependências** certas para cada tipo de projeto
3. Entender a **estrutura de pastas** de um projeto Maven
4. Conhecer a classe com `@SpringBootApplication`
5. Criar e testar o primeiro endpoint `GET`

Antes de começar

- Aula 01: JDK 25 e VS Code configurados

Duração sugerida

100 min

Código da disciplina

Projetos de referência na pasta PROJETOS do GitHub

Roteiro

01 O que é o Spring Boot

02 Gerando o projeto pelo site

03 Gerando pelo VS Code

04 Estrutura de pastas

05 O primeiro endpoint

06 Formas de rodar o projeto

07 Aprofundando: IoC, beans e autoconfiguração

01

O que é o Spring Boot

O que é o Spring Boot

O **Spring** é o framework Java mais usado no mercado para back-end. Ele é grande e modular: Spring MVC (web), Spring Data (bancos), Spring Security (segurança) e muitos outros. O **Spring Boot** é a camada que deixa tudo isso fácil de usar:

- **Starters**: uma dependência como `spring-boot-starter-webmvc` já traz tudo que um projeto web precisa.
- **Autoconfiguração**: o Spring olha as bibliotecas no projeto e configura sozinho (achou o driver do PostgreSQL? cria a conexão).
- **Servidor embutido**: o Tomcat vem dentro do `.jar`. Basta `java -jar app.jar` para subir.

01

O que é o Spring Boot

Aula 02 · Back-end

O que é o Spring Boot

Usaremos o **Spring Boot 4.1** (a linha 4.x foi lançada em novembro de 2025). Se você encontrar material da linha 3.x, quase tudo vale igual; as diferenças mais visíveis estão destacadas ao longo das aulas.

01

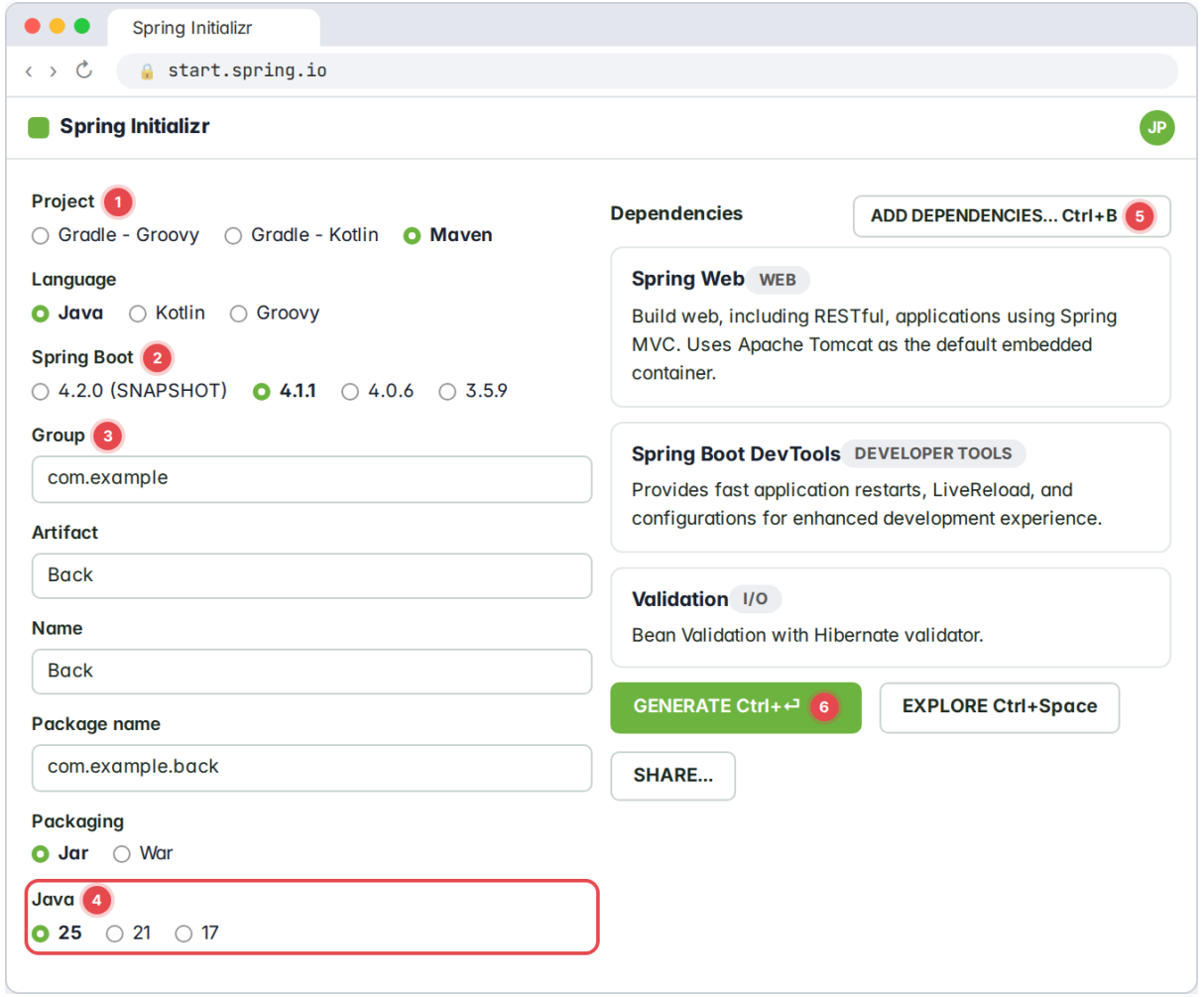
O que é o Spring Boot

Aula 02 · Back-end

02

Gerando o projeto pelo site

Gerando o projeto pelo site



Acesse **start.spring.io** e preencha conforme a figura. Os números indicam a ordem do passo a passo.

Tela do Spring Initializr preenchida para um projeto de API REST com Java 25.

Gerando o projeto pelo site

1. **Project: Maven.** Usaremos Maven em todos os projetos (o arquivo de configuração é o `pom.xml`).
2. **Spring Boot:** escolha a versão estável mais recente (sem *SNAPSHOT* ou *M1/RC*).
3. **Group / Artifact:** o *group* é o domínio invertido da empresa (`com.empresa`); o *artifact* é o nome do projeto. O *package name* é montado com os dois.
4. **Java: 25.** Precisa bater com o JDK instalado.
5. **Add dependencies:** procure e adicione **Spring Web**, **Spring Boot DevTools** e **Validation**. Para banco, adicione depois *Spring Data JPA* + *PostgreSQL Driver* ou *Spring Data MongoDB* (aulas 06 e 07).

Gerando o projeto pelo site (cont.)

6

Generate: baixa um `.zip`. Extraia e abra a pasta no VS Code (`code Back`).

Dependências mais usadas nos projetos

No Initializr	Artefato no `pom.xml`	Quando usar
Spring Web	`spring-boot-starter-webmvc` (*no Boot 3: `-web`*)	Toda API REST
Thymeleaf	`spring-boot-starter-thymeleaf`	Páginas HTML geradas no servidor (aula 08)
Spring Data JPA	`spring-boot-starter-data-jpa`	Bancos relacionais (Supabase/PostgreSQL)
PostgreSQL Driver	`org.postgresql:postgresql`	Conectar ao Supabase
Spring Data MongoDB	`spring-boot-starter-data-mongodb`	MongoDB Atlas

Dependências mais usadas nos projetos (cont.)

No Initializr	Artefato no `pom.xml`	Quando usar
Spring Security	`spring-boot-starter-security`	Login, senhas, JWT (aulas 09 e 10)
Validation	`spring-boot-starter-validation`	`@NotBlank`, `@Email` nos DTOs
Spring Boot DevTools	`spring-boot-devtools`	Reinicia sozinho ao salvar
Lombok	`org.projectlombok:lombok`	Gera getters/setters por anotação (opcional)

03

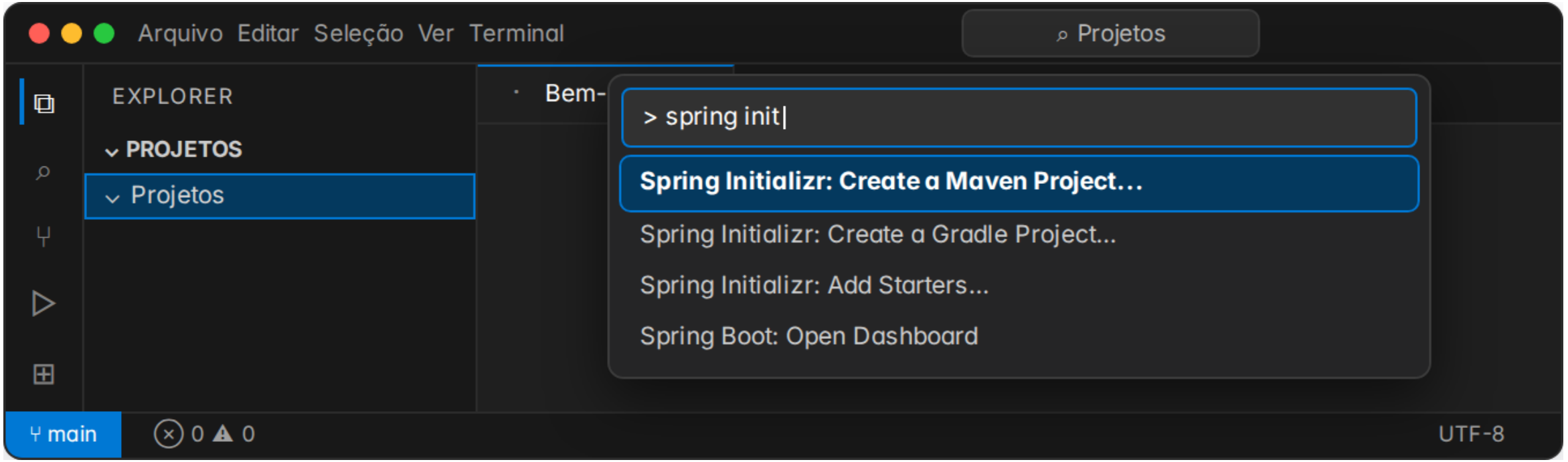
Gerando pelo VS Code

Gerando pelo VS Code

Com o *Spring Boot Extension Pack* instalado dá para fazer tudo sem sair do editor:

1. `Ctrl+Shift+P` e digite **Spring Initializr: Create a Maven Project…**
2. Escolha a versão do Spring Boot, **Java** como linguagem, o *group id* (`com.example`) e o *artifact id* (`Back`).
3. Escolha **Jar** e a versão **25** do Java.
4. Marque as dependências (espaço seleciona) e pressione Enter.
5. Escolha a pasta de destino e clique em **Open** quando o VS Code perguntar.

Gerando pelo VS Code

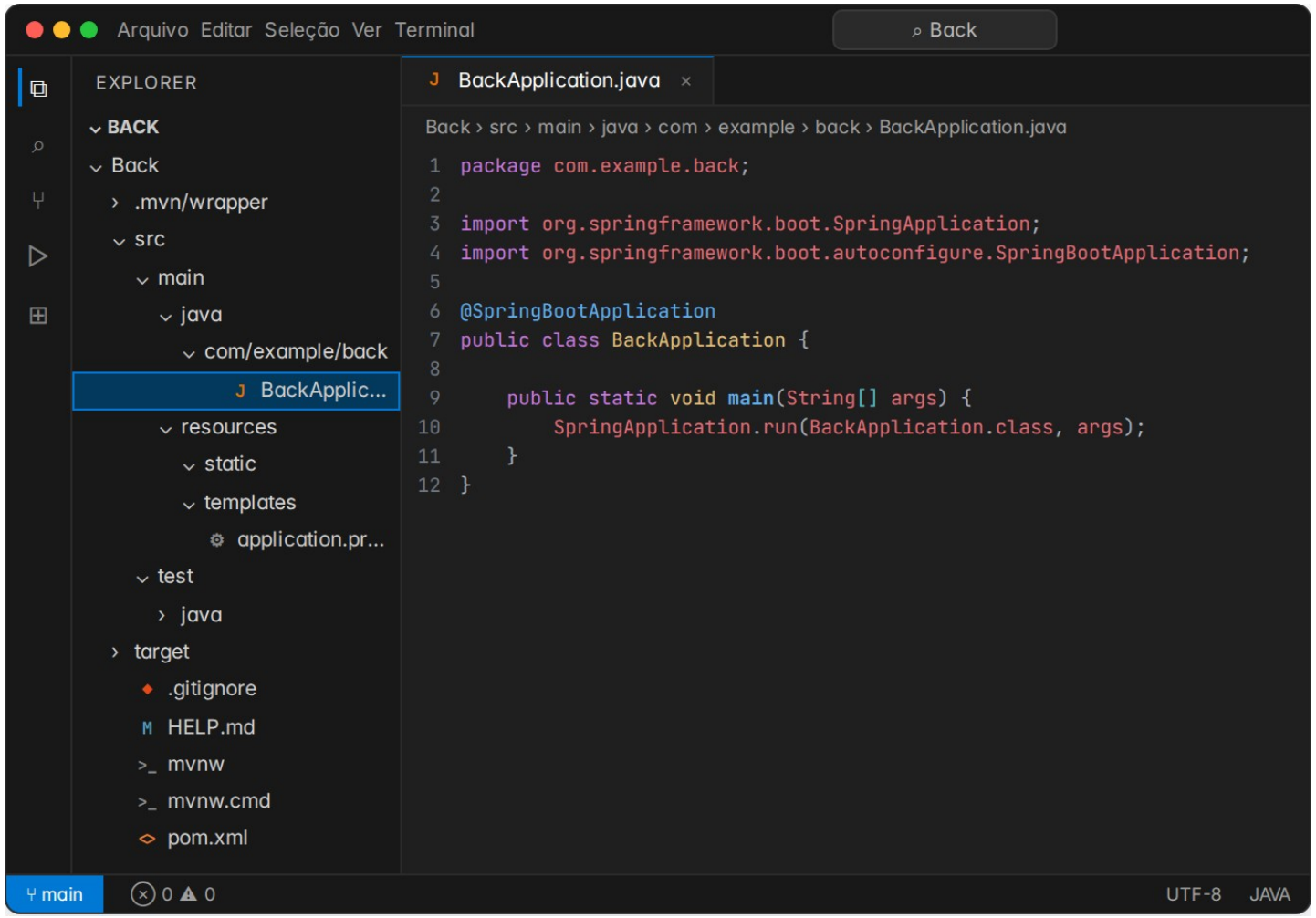


Paleta de comandos (`Ctrl+Shift+P`) filtrada por *spring init*: escolha **Create a Maven Project**.

04

Estrutura de pastas

Estrutura de pastas



O Maven impõe uma estrutura padrão. Saber o que vai em cada lugar evita muita confusão:

Projeto recém-gerado aberto no VS Code: a classe principal fica no pacote raiz `com.example.back`.

Estrutura de pastas

Caminho	O que guarda
`src/main/java`	Todo o código Java, organizado em pacotes
`src/main/resources`	Arquivos que não são Java: `application.properties`, `static/` (CSS, JS, imagens), `templates/` (Thymeleaf)
`src/test/java`	Testes automatizados (JUnit)
`target/`	Gerada pelo Maven: classes compiladas e o `.jar`. **Nunca vai para o Git.**
`pom.xml`	Configuração do projeto e lista de dependências (aula 03)
`mvnw`, `mvnw.cmd`, `.mvn/`	Maven Wrapper

A anotação @SpringBootApplication

Ela junta três anotações em uma: `@Configuration` (a classe pode declarar *beans*), `@EnableAutoConfiguration` (liga a autoconfiguração) e `@ComponentScan` (procura classes anotadas **no mesmo pacote e nos subpacotes**). Por isso todos os pacotes (`controller`, `service`…) devem ficar *abaixo* de `com.example.back`.

04

Estrutura de pastas

Aula 02 · Back-end

A anotação @SpringBootApplication

Variação usada nos projetos

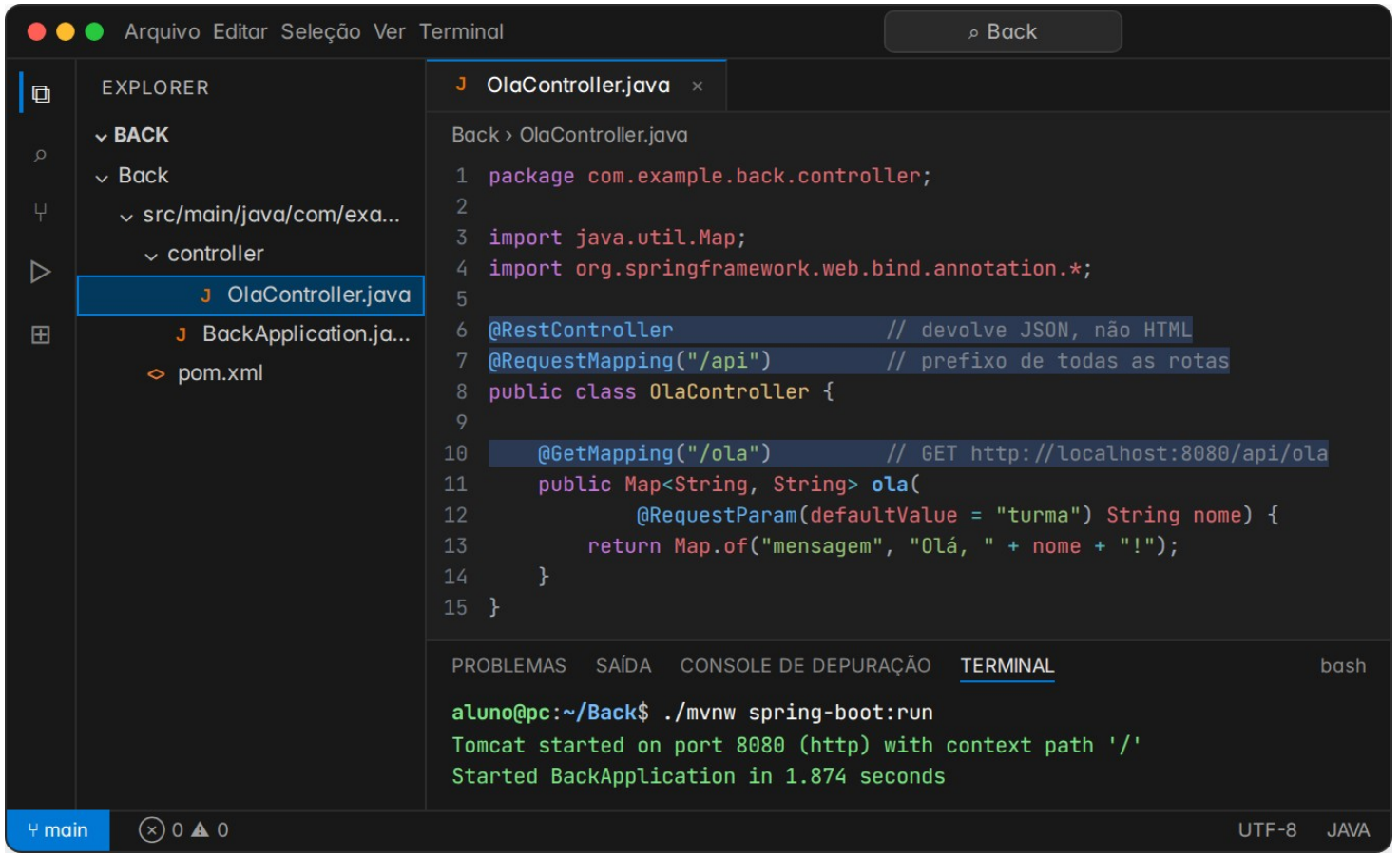
Nos projetos da disciplina a classe principal fica em um pacote próprio, `com.example.back.application`. Como ela não está mais no pacote raiz, avisamos ao Spring onde procurar:

```
@SpringBootApplication(scanBasePackages = "com.example.back")
@EnableMongoRepositories("com.example.back.repository")   // ou @EnableJpaRepositories
@EntityScan("com.example.back.model")                      // só para JPA
```

05

O primeiro endpoint

O primeiro endpoint



Crie o pacote `controller` e, dentro dele, a classe `OlaController`:

Controller com um endpoint GET. O terminal mostra o Tomcat subindo na porta 8080.

O primeiro endpoint



Resposta JSON do endpoint no navegador.

O primeiro endpoint

Erro comum

`Port 8080 was already in use`: já existe outro programa (talvez outra execução do mesmo projeto) usando a porta. Pare o processo antigo com `Ctrl+C` no terminal onde ele roda, ou troque a porta com `server.port=8081` no `application.properties`.

06

Formas de rodar o projeto

Formas de rodar o projeto

Forma	Como	Quando usar
Terminal	`./mvnw spring-boot:run`	Sempre funciona; é o que o professor usa nos exemplos
VS Code	Botão **Run** acima do `main` ou *Spring Boot Dashboard*	Para depurar com *breakpoints*
`.jar`	`./mvnw clean package` e depois `java -jar target/Back-0.0.1-SNAPSHOT.jar`	Simula a produção; é o que o Docker faz

Formas de rodar o projeto

Com o **DevTools** no projeto, ao salvar um arquivo a aplicação reinicia sozinha em poucos segundos.

06

Formas de rodar o projeto

Aula 02 · Back-end

07

Aprofundando: IoC, beans e autoconfiguração

Aprofundando: IoC, beans e autoconfiguração

O coração do Spring é a **Inversão de Controle** (IoC): em vez de o seu código criar os objetos com `new`, o Spring cria, guarda e entrega esses objetos. Cada objeto gerenciado pelo Spring se chama **bean**, e o lugar onde eles vivem é o **ApplicationContext**.

07

Aprofundando: IoC, beans e autoconfiguração

Aula 02 · Back-end

Aprofundando: IoC, beans e autoconfiguração

Anotação	Papel	Pacote típico
`@Component`	Bean genérico	`dao`, `mapper`, `facade`
`@Service`	Regra de negócio	`service`
`@Repository`	Acesso a dados (traduz exceções do banco)	`dao`
`@Controller`	Devolve páginas (Thymeleaf)	`controller`
`@RestController`	Devolve JSON	`controller`
`@Configuration` + `@Bean`	Cria beans manualmente	`config`

Aprofundando: IoC, beans e autoconfiguração

config/AppConfig.java

```java
@Configuration
public class AppConfig {

    @Bean                          // o retorno vira um bean
    public Clock relogio() {
        return Clock.system(ZoneId.of("America/Sao_Paulo"));
    }

    @Bean                          // roda uma vez, quando a aplicação sobe
    public CommandLineRunner aoIniciar(Clock relogio) {
        return args -> System.out.println("API iniciada em " + LocalDateTime.now(relogio));
    }
}
```

@Controller x @RestController

@Controller

- O retorno é o **nome de um template**
- Usado com Thymeleaf
- Para JSON precisa de `@ResponseBody`

vs

@RestController

- O retorno é **convertido em JSON**
- Usado em APIs consumidas pelo React
- = `@Controller` + `@ResponseBody`

O que a autoconfiguração faz por você

- Achou `spring-boot-starter-webmvc`? Sobe um **Tomcat** na porta 8080 e configura o Jackson para JSON.
- Achou o driver do PostgreSQL e `spring.datasource.url`? Cria o **pool de conexões** (HikariCP).
- Achou `spring-boot-starter-security`? Protege **todas** as rotas e gera um usuário temporário.
- Achou `spring-boot-starter-thymeleaf`? Procura templates em `src/main/resources/templates`.

07

Aprofundando: IoC, beans e autoconfiguração

Aula 02 · Back-end

O que a autoconfiguração faz por você

Relatório de autoconfiguração

```
# mostra quais autoconfigurações foram aplicadas e por quê
./mvnw spring-boot:run -Dspring-boot.run.arguments=--debug

# trecho da saída
============================
CONDITIONS EVALUATION REPORT
============================
Positive matches:
   DataSourceAutoConfiguration matched:
      - @ConditionalOnClass found required classes 'javax.sql.DataSource'
Negative matches:
   MongoAutoConfiguration:
      Did not match: @ConditionalOnClass did not find 'com.mongodb.client.MongoClient'
```

Recursos que vêm “de graça” com o Spring Boot

DevTools

Reinício automático ao salvar e LiveReload do navegador.

Actuator

`/actuator/health`, `/actuator/metrics`: saúde e métricas da aplicação (Deploy, aula 10).

Logging

Logback configurado; ajuste com `logging.level.*`.

Virtual threads

`spring.threads.virtual.enabled=true` com Java 21+: mais requisições simultâneas com menos memória.

Aprofundando: IoC, beans e autoconfiguração

Dica

Para trocar o banner do console, crie `src/main/resources/banner.txt` com o texto (ou arte ASCII) que quiser. Para desligar: `spring.main.banner-mode=off`.

Aprofundando: IoC, beans e autoconfiguração

Estrutura comum de logs	Exemplo
Data e hora	`2026-11-03T10:15:22.418-03:00`
Nível	`INFO`, `WARN`, `ERROR`, `DEBUG`
PID e thread	`12345 --- [main]`
Classe que logou	`o.s.b.w.embedded.tomcat.TomcatWebServer`
Mensagem	`Tomcat started on port 8080 (http)`

Aprofundando: IoC, beans e autoconfiguração

Usando o logger em vez de System.out.println

```
@Service
public class AlunoService {
    private static final Logger log = LoggerFactory.getLogger(AlunoService.class);

    public Aluno inserir(Aluno a) {
        log.info("Inserindo aluno {}", a.getEmail());
        // ...
    }
}
```

08

Quiz de revisão

Quiz · questão 1 de 5

Pergunta

O que a anotação `@SpringBootApplication` faz?

A. Só define a porta

B. Junta @Configuration, @EnableAutoConfiguration e @ComponentScan

C. Cria o banco de dados

D. Gera o pom.xml

Quiz · questão 1 de 5

Resposta

O que a anotação `@SpringBootApplication` faz?

A. Só define a porta

✓ Junta @Configuration, @EnableAutoConfiguration e @ComponentScan

C. Cria o banco de dados

D. Gera o pom.xml

Por quê: Ela liga a autoconfiguração e procura componentes no próprio pacote e nos subpacotes.

Pergunta

Se a classe principal está em `com.example.back.application`, o que é preciso fazer?

A Nada

B Informar `scanBasePackages = "com.example.back"`

C Renomear o projeto

D Apagar o pacote controller

Resposta

Se a classe principal está em `com.example.back.application`, o que é preciso fazer?

A Nada

✓ Informar `scanBasePackages = "com.example.back"`

C Renomear o projeto

D Apagar o pacote controller

Por quê: Sem isso o Spring só procuraria beans dentro de `application`.

Quiz · questão 3 de 5

Pergunta

Qual a diferença entre `@RestController` e `@Controller`?

A Nenhuma

B @RestController devolve JSON; @Controller devolve o nome de um template

C @Controller é mais rápido

D @RestController só aceita GET

Resposta

Qual a diferença entre `@RestController` e `@Controller`?

A Nenhuma

✓ @RestController devolve JSON; @Controller devolve o nome de um template

C @Controller é mais rápido

D @RestController só aceita GET

Por quê: @RestController = @Controller + @ResponseBody.

Pergunta

No Spring Boot 4, o starter para APIs REST se chama:

A spring-boot-starter-rest

B spring-boot-starter-webmvc

C spring-boot-starter-api

D spring-boot-starter-json

Resposta

No Spring Boot 4, o starter para APIs REST se chama:

A spring-boot-starter-rest

✓ spring-boot-starter-webmvc

C spring-boot-starter-api

D spring-boot-starter-json

Por quê: No Boot 3 o nome era `spring-boot-starter-web`.

Quiz · questão 5 de 5

Pergunta

O que acontece com o erro `Port 8080 was already in use`?

A O banco está fora do ar

B Outro processo já usa a porta 8080

C Falta o JDK

D O pom.xml está inválido

Quiz · questão 5 de 5

Resposta

O que acontece com o erro `Port 8080 was already in use`?

A. O banco está fora do ar

✓ Outro processo já usa a porta 8080

C. Falta o JDK

D. O pom.xml está inválido

Por quê: Pare a execução anterior ou troque `server.port`.

Fechamento

Resumo da aula

- ✓ O **Spring Initializr** (site ou VS Code) gera o projeto com `pom.xml`, Maven Wrapper e classe principal.
- ✓ Escolha **Maven**, **Java 25**, **Jar** e os *starters* que o projeto precisa.
- ✓ `@SpringBootApplication` faz o *component scan* a partir do próprio pacote.
- ✓ `@RestController` + `@GetMapping` criam um endpoint que devolve JSON.
- ✓ Rode com `./mvnw spring-boot:run`; a API sobe em `http://localhost:8080`.

Exercícios

1. Gere um projeto `Biblioteca` com Spring Web e DevTools e rode-o.

2. Crie o endpoint `GET /api/hora` que devolve a data e a hora atuais em JSON.

3. Crie `GET /api/soma?a=2&b=3` que devolve `{ "resultado": 5 }`.

4. Mova a classe principal para um pacote `application` e faça o projeto voltar a funcionar usando `scanBasePackages`.

Para ir além

- start.spring.io
- docs.spring.io/spring-boot: documentação oficial
- Projetos de referência: `PROJETOS/SpringBoot/RestAPI_v1`, `RestAPI_v2` e `RestAPI_v3`

Repositório da disciplina

Todos os projetos citados estão na pasta PROJETOS

Abrir no GitHub

Links e versões citados podem ter mudado. Confira sempre a documentação oficial.



Obrigado!

Dúvidas sobre a aula 02?

Prof. Dr. João Paulo Aramuni
Engenharia de Software · PUC Minas
Desenvolvimento e Integração de Aplicações Web

GitHub da disciplina
https://github.com/joaopauloaramuni/desenvolvimento-e-integracao-de-aplicacoes-web